

E03 — Learning Rate Calibration: Muon vs SGD

Overview

This experiment investigates how **learning rate** affects the convergence performance of Muon and SGD on the matrix sensing problem. Finding the optimal learning rate is critical for fair algorithm comparison.

Why this matters: Muon's spectral normalization changes the effective step size, so its optimal learning rate may differ from SGD's. This experiment systematically scans learning rates to identify the best operating point for each algorithm.

Experiment ID: E03 | **Problem Domain:** Matrix Sensing | **Algorithms:** Muon-Exact vs SGD

Scientific Question

Hypothesis

Muon and SGD have different optimal learning rates due to Muon's spectral normalization. Muon may perform well at smaller learning rates where SGD struggles, or vice versa.

Key Metrics

- ϵ : Iterations to reach $\epsilon = 0.01$ (primary; lower is better)
- min_loss**: Best loss achieved
- final_loss**: Loss after 2000 iterations
- ϵ : Total FLOPs

Statistical Framework

Paired t-tests for each (dimension, learning rate) combination. Identify lr minimizing mean ϵ per algorithm.

Experimental Design

Parameter	Value	Description
d	{50, 70}	Matrix dimensions

Parameter	Value	Description
r	5	Target rank
m	dr	Measurements
lr	{0.001, 0.003, 0.01, 0.03, 0.1}	Learning rates scanned
noise	0.0	No noise
dist	normal	Gaussian matrices
Iterations	2000	Max iterations
Seeds	5	Replications per config
ϵ	0.01	Convergence threshold

Total runs: 2 algos x 2 dims x 5 lrs x 5 seeds = 100 runs

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
plt.style.use('seaborn-v0_8-whitegrid')
df = pd.read_csv('../results_v3/E03_detailed_results.csv')
print(f"Shape: {df.shape}")
print(f"Algorithms: {df['algo'].unique()}")
print(f"Dimensions: {sorted(df['d'].unique())}")
print(f"Learning rates: {sorted(df['lr'].unique())}")
df.head()
```

Exploratory Data Analysis

```
In [ ]: pivot = df.groupby(['algo', 'd', 'lr'])['K_epsilon'].agg(['mean', 'std']).reset_index()
print(pivot)
print("
Best lr per (algo, d):")
best_lr = df.groupby(['algo', 'd', 'lr'])['K_epsilon'].mean().reset_index()
for algo in best_lr['algo'].unique():
    for d in sorted(best_lr[best_lr['algo']==algo]['d'].unique()):
        subset = best_lr[(best_lr['algo']==algo)&(best_lr['d']==d)]
        best = subset.loc[subset['K_epsilon'].idxmin()]
        print(f" {algo} d={d}: best lr={best['lr']:.4f}, mean K_eps={best['K_epsilon']:.4f}")
```

Visualization 1: K_ϵ vs Learning Rate (Log Scale)

```
In [ ]: fig, axes = plt.subplots(1, 2, figsize=(14, 6))
dims = sorted(df['d'].unique())
lrs = sorted(df['lr'].unique())
for idx, d in enumerate(dims):
```

```

ax = axes[idx]
for algo, color in [('Muon-Exact', '#2E86AB'), ('SGD', '#F18F01')]:
    subset = df[(df['algo']==algo)&(df['d']==d)]
    means = [subset[subset['lr']==lr]['K_epsilon'].mean() for lr in lrs]
    stds = [subset[subset['lr']==lr]['K_epsilon'].std() for lr in lrs]
    ax.errorbar(lrs, means, yerr=stds, label=algo, color=color, marker='o')
ax.set_xscale('log')
ax.set_xlabel('Learning Rate (log scale)', fontsize=12)
ax.set_ylabel('Average K_epsilon (iterations)', fontsize=12)
ax.set_title(f'd={d}', fontsize=13, fontweight='bold')
ax.legend(); ax.grid(alpha=0.3)
fig.suptitle('E03: Convergence Speed vs Learning Rate', fontsize=15, fontweight='bold')
plt.tight_layout(); plt.show()

```

Visualization 2: Heatmap of K_ϵ

```

In [ ]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for idx, algo in enumerate(['Muon-Exact', 'SGD']):
    subset = df[df['algo']==algo]
    pivot_data = subset.groupby(['d', 'lr'])['K_epsilon'].mean().unstack()
    cbar_label = 'Mean K_epsilon'
    sns.heatmap(pivot_data, annot=True, fmt='.0f', cmap='YlOrRd',
                ax=axes[idx], cbar_kws={'label': cbar_label})
    axes[idx].set_title(algo, fontsize=13, fontweight='bold')
    axes[idx].set_xlabel('Learning Rate', fontsize=11)
    axes[idx].set_ylabel('Dimension d', fontsize=11)
fig.suptitle('E03: Heatmap of Average K_epsilon', fontsize=15, fontweight='bold')
plt.tight_layout(); plt.show()

```

Visualization 3: Best Learning Rate Comparison

```

In [ ]: fig, ax = plt.subplots(figsize=(10, 6))
best_data = []
for algo in ['Muon-Exact', 'SGD']:
    for d in dims:
        subset = df[(df['algo']==algo)&(df['d']==d)]
        lr_means = {lr: subset[subset['lr']==lr]['K_epsilon'].mean() for lr in lrs}
        best_lr = min(lr_means, key=lr_means.get)
        best_k = lr_means[best_lr]
        best_data.append({'algo': algo, 'd': d, 'best_lr': best_lr, 'best_K': best_k})
best_df = pd.DataFrame(best_data)
for algo, color in [('Muon-Exact', '#2E86AB'), ('SGD', '#F18F01')]:
    subset = best_df[best_df['algo']==algo]
    ax.plot(subset['d'], subset['best_lr'], marker='o', markersize=12, color=color)
ax.set_xlabel('Dimension d', fontsize=12)
ax.set_ylabel('Optimal Learning Rate', fontsize=12)
ax.set_title('E03: Optimal Learning Rate by Dimension', fontsize=14, fontweight='bold')
ax.set_yscale('log'); ax.legend(); ax.grid(alpha=0.3)
plt.tight_layout(); plt.show()
print(best_df)

```

Statistical Tests

```
In [ ]: print("Paired t-tests: Muon vs SGD per (d, lr)")
print("-" * 80)
sig_count = 0
total = 0
for d in sorted(df['d'].unique()):
    for lr in sorted(df['lr'].unique()):
        muon_k = df[(df['algo']=='Muon-Exact') & (df['d']==d) & (df['lr']==lr)].
        sgd_k = df[(df['algo']=='SGD') & (df['d']==d) & (df['lr']==lr)].sort_val
        if len(muon_k) > 1 and len(muon_k) == len(sgd_k):
            t_stat, p_val = stats.ttest_rel(muon_k, sgd_k)
            sig = "***" if p_val < 0.001 else "*" if p_val < 0.01 else ""
            if p_val < 0.05: sig_count += 1
            total += 1
            print(f"d={d:>3d} lr={lr:>6.4f}: t={t_stat:>+6.3f} p={p_val:.4f}")
print(f"Significant differences: {sig_count}/{total} configs")
```

Conclusions & Interpretation

Key Findings

1. Optimal Learning Rate Range:

- **Muon** performs best at moderate learning rates ($lr \sim 0.003-0.01$), where spectral normalization provides stable updates.
- **SGD** achieves excellent convergence across a broader range, with best performance at $lr \sim 0.01-0.03$.

2. Sensitivity to Large Learning Rates:

- At $lr = 0.03$ and 0.1 , **Muon fails to converge** within 2000 iterations ($K_{\epsilon} = 2001, I_{\text{conv}} = 0$). The aggressive spectral normalization combined with large step sizes causes instability.
- **SGD remains stable** even at $lr = 0.1$, achieving convergence in all configurations.

3. Robustness Profile:

- **SGD is more robust** to learning rate variations.
- **Muon has a narrower optimal range** but excels within it, often requiring fewer iterations at comparable learning rates.

Recommendations

- For Muon: use lr in $[0.003, 0.01]$; avoid $lr > 0.02$.
- For SGD: use lr in $[0.01, 0.03]$; the algorithm is forgiving of suboptimal choices.

- When comparing algorithms, ensure both use their respective optimal learning rates.